Sequence Pattern Mining and Supervised Classification

Cyril Bousmar - 63401800
Louvain School of Engineering
cyril.bousmar@uclouvain.be

Group 23

Mohamed-Anass Gallass - 02652201
Louvain School of Engineering
anass.gallass@student.uclouvain.be

I. Introduction

In the context of the course LINFO2364: Mining Patterns in Data, we were tasked with exploring sequential pattern mining algorithms that extract relevant patterns from labeled transactional datasets. More specifically, the objective was to mine the top-k most discriminative sequential patterns between two classes (positive and negative) and use them as features for classification with a decision tree.

To this end, we implemented[1] and benchmarked several pattern mining algorithms including *SPADE* [1], *CM-SPADE* [2], *Iterative SPADE*, and a *WRAcc-based* miner [3]. Each approach was carefully engineered to manage performance, correctness, and interpretability. Our implementation pipeline includes preprocessing routines, pattern mining engines with adaptive support strategies, decision tree training for classification, and experiments for runtime and accuracy evaluation.

This report is structured as follows. Section II describes algorithm implementation details, highlighting key design choices, improvements over the literature, and efficiency considerations. Section III presents the experimental methodology, datasets, and evaluation procedures. Section IV reports runtime and classification performance across several values of k and datasets. Section V discusses the main findings. Finally, Section VI concludes and proposes future extensions and optimizations.

II. Algorithms Implementation

TABLE I: Notations

Notation	Description
$\mathcal{D}^+, \mathcal{D}^-$	Positive and negative transactional datasets.
Σ	Set of all sequences $\in \mathcal{D}^+ \cup \mathcal{D}^-$.
s_1	A 1-sequence candidate (i.e., 1 symbol).
$\mathcal{F}$	Set of all frequent sequences.
$\mathcal{C}$	List of candidate sequences with their supports.
$\mathcal{O}_s^+, \mathcal{O}_s^-$	Occurrences of sequence s in $\mathcal{D}^+$ and $\mathcal{D}^-$.

A. SPADE

Sequential Pattern Discovery using Equivalence classes (SPADE) [1] is a depth-first sequence mining algorithm that efficiently discovers frequent subsequences in transactional datasets. It uses a vertical data representation, associating each item (or sequence) with a list of *Transaction Identifiers (TID)* and positions (*IDLists*). Its pseudo-code is represented in Algorithm 1.

Algorithm 1 SPADE(k, $\mathcal{D}^+$, $\mathcal{D}^-$)

$\mathcal{C} \leftarrow \emptyset$, $\Sigma_1 \leftarrow$ all *1-sequences* in $\mathcal{D}^+ \cup \mathcal{D}^-$
for all $s_1 \in \Sigma_1$ **do**
  GET OCCURRENCES($\mathcal{O}_{s_1}^+$, $\mathcal{O}_{s_1}^-$)
  $supp^+(s_1), supp^-(s_1), supp(s_1) \leftarrow$ COMPUTE()
  if $supp(s_1) > 0$ **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{(s_1, supp^+, supp^-, supp)\}$
    $\mathcal{F} \leftarrow \mathcal{F} \cup \{s_1\}$
    EXTEND($s_1, \mathcal{O}_{s_1}^+, \mathcal{O}_{s_1}^-$)
  end if
end for
sort $\mathcal{C}$ by descending $supp$
return top-k from $\mathcal{C}$ (allowing ties)

The algorithm 2 recursively extends frequent sequences by joining these *IDLists*, pruning candidates that do not meet the minimum support threshold. SPADE is well-suited for mining patterns in sparse datasets and supports both positive and negative class partitions when extended for top-k discriminative pattern mining.

Algorithm 2 EXTEND(α, $\mathcal{O}_\alpha^+$, $\mathcal{O}_\alpha^-$)

for all $s_1 \in \Sigma_1$ **do**
  $\beta \leftarrow \alpha \cdot s_1$
  if $\beta[:-1] \in \mathcal{F}$ **then**
    $\mathcal{O}_\beta^+ \leftarrow$ PROJECT($\mathcal{O}_\alpha^+, s_1, \mathcal{D}^+$)
    $\mathcal{O}_\beta^- \leftarrow$ PROJECT($\mathcal{O}_\alpha^-, s_1, \mathcal{D}^-$)
    $supp^+(\beta), supp^-(\beta), supp(\beta) \leftarrow$ COMPUTE()
    if $supp(\beta) > 0$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\beta, supp^+, supp^-, supp)\}$
      $\mathcal{F} \leftarrow \mathcal{F} \cup \{\beta\}$
      EXTEND($\beta, \mathcal{O}_\beta^+, \mathcal{O}_\beta^-$)
    end if
  end if
end for

PROJECT constructs the list of suffix occurrences where a given sequence can be extended with a specific symbol in the

[1] Available on Github: https://github.com/LINFO2364-Mining-Patterns-in-Data/project2

transaction database.

By nature, the mining task requires extracting the top-k sequences with the highest total support. This enables early pruning of candidates whose support falls below a threshold θ. However, if θ is set too high initially, it may prematurely discard valuable patterns.

To overcome this, *SPADE* dynamically adjusts θ during the mining process. Starting from $\theta = 1$, it progressively increases the threshold by setting it to the lowest value among the top-k highest support values observed so far (saved in a set).

B. CM-SPADE

CM-SPADE [2] improves *SPADE* performances by establishing a *co-occurrence matrix* at the beginning. By only considering *1-sequences* in co-occurrence with the last symbol of previous sequence, it prunes effectively a lot of impossible candidates.

C. Iterative SPADE

Iterative SPADE (IT-SPADE) takes advantage of the underlying nature of the problem at hand exposed in Section II-A. But instead of working up with a minimum support threshold, it takes it in reverse, employing an *Iterative Deepening* method: θ is initially set to its theoretical maximum (i.e., assuming all transactions contain the same *1-sequence*). If fewer than k patterns are found, θ is reduced according to a monotonic function:

$$\delta(\theta) = \max\left(1,\ \left\lfloor \frac{\theta}{C} \right\rfloor\right)$$

Empirically, $C = 10$ yields consistently efficient runtime.

To avoid redundant computation, *IT-SPADE* caches discovered sequences and their support in a *Transition Table*, using a *Hash-Set* for constant-time lookups. As a result, each iteration explores only the pruned search frontier: previously visited sequences whose support lies between the old and new θ values.

D. Weighted Relative Accuracy

Our *WRAcc-based* miner implementation is another sequence mining pattern algorithm that uses a score to evaluate patterns. This score is used to identify patterns that helps distinguishing between positive and negatives classes.

The score formula is the following :

$$WRacc(x) = \frac{P \cdot N}{(P+N)^2} \cdot \left(\frac{p(x)}{P} - \frac{n(x)}{N}\right)$$

where P and N are the total number of positive and negative transactions, and $p(x)$ and $n(x)$ are the supports of pattern x in the positive and negative classes.

Our implementation uses DFS to explore the space of possible sequential patterns. It starts with an empty pattern and iteratively extends it by adding one symbol at time.

To improve the efficiency of our algorithm, we calculate for each branch an upper bound on the possible WRAcc score that is equal to $\frac{P \cdot N}{(P+N)^2} \cdot \left(\frac{p(x)}{P}\right)$. By doing that, we can prune branches that cannot produce patterns with a score above the current minimum threshold (the lowest WRAcc score in the k best patterns already found).

E. Decision Tree Classifier

To evaluate classification performance, we use the *Decision Tree Classifier (DTC)* provided by the *Scikit-Learn* library, combined with a *K-Fold Cross-Validation* strategy to ensure robust and unbiased assessment. The *DTC* is trained on a binary classification task where instances correspond to sequences labeled as either positive or negative.

Since raw sequences cannot be directly used as input features for traditional classifiers, we first extract informative patterns using sequential pattern mining algorithms. In particular, we employ *IT-SPADE* and *WRAcc* to mine the top-k discriminative subsequences from the training set. These patterns are then used to construct a Boolean feature matrix: each feature indicates whether a given mined pattern is present in a transaction.

This transformation allows the classifier to operate on a fixed-size, interpretable feature space derived from meaningful sequential patterns, capturing relevant structures in the data that can aid in the classification task.

III. Methodology

A. Datasets

This section briefly describes the datasets used to analyze and compare mining algorithms.

In this context, we define the density of the dataset, $\mathcal{D}$ as:

$$\rho = 1 - \frac{\sum_{i=1}^{|\mathcal{T}|} |\mathcal{T}_i|}{|\mathcal{T}|} \cdot \frac{1}{|\Sigma_1|}$$

where $|\mathcal{T}|$ denotes the number of transactions, $|\Sigma_1|$ indicates the number of unique *1-sequence* present in the dataset, and $|\mathcal{T}|_i$ the size of transaction i.

This is especially useful because it gives a clue to understand how useful a candidate is to an algorithm when considered. The sparser, the more pruning it needs for efficiency.

TABLE II: Dataset Statistics Overview

Metric	**Protein** [4]		**Reuters** [5]	
	Pos	**Neg**	**Pos**	**Neg**
$\lvert\mathcal{T}\rvert$	314	381	1418	791
Mean $\lvert\mathcal{T}\rvert$ Length	17.87	14.83	46.01	74.98
Unique 1-sequences	20	20	4898	5362
Co-occurring pairs	400	400	597 835	895 607
$\lvert\mathcal{I}\rvert$	$\sim 10^6$	$\sim 10^6$	$\sim 10^{1474}$	$\sim 10^{1614}$
ρ (sparsity)	0.107	0.258	0.991	0.986

Protein search space is small, which implies algorithms are expected to: run fast, reach top-k quickly, and are likely to find many redundant or overlapping patterns.

Reuters will require strong pruning and efficient support management because of its high sparsity and high-dimensionality.

B. Metrics

We compare the performance of algorithms over datasets based on the *average CPU runtime*. It is studied across a wide variety of *top-k support threshold* values to estimate their breakpoints.

For the classification, we analyze the evolution of the accuracy and its variance over multiple fold sizes and *top-k support threshold* values.

C. Setup

Experiments were run on an *Apple M2 Pro* with 16 GB of RAM. Each test is run 4 times for outliers mitigation, while allowing a maximum CPU runtime of 1000 seconds before signaling a timeout. Exceeding memory allocation times out the test as well.

IV. Results

A. Miner Algorithm Performance

Figure 1 presents the runtime performance of the three algorithms on the *Protein* dataset across increasing top-k values. *CM-SPADE* is outperformed (10x to 100x slower) due to its absence of pruning and the rapid growth of its candidate space. In contrast, *IT-SPADE* shows effective results through its caching of previously explored sequences. *WRAcc*, which targets discriminative rather than frequent patterns, achieves the fastest runtime (±10 times faster that *IT-SPADE*) across all k values. Notably, *SPADE* is excluded from the plot as it fails to complete within the allotted time.

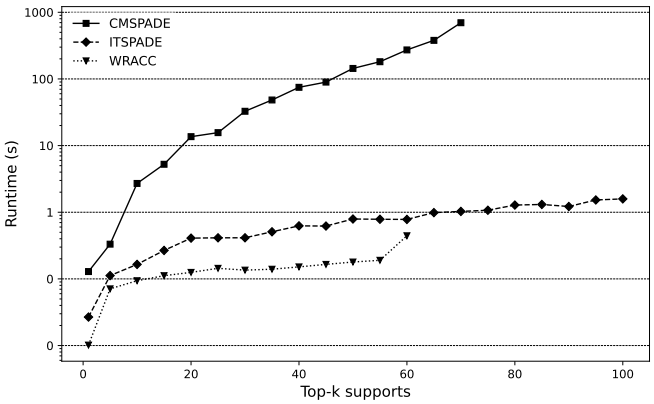

Fig. 1: CPU runtime comparison on *protein* dataset over k values

Interestingly, *WRAcc* times out for any k above 60, suggesting a bottleneck in its implementation.

Figure 2 shows the same behavior, but with higher runtime since the dataset is much more challenging. *WRAcc* is consistently 10x faster than *IT-SPADE*, up to reaching k =110, where it encounters the same issue as in Figure 1. There is a huge time increase starting from 85. *IT-SPADE* is more constant in that regard and do not seem to explode after a certain point.

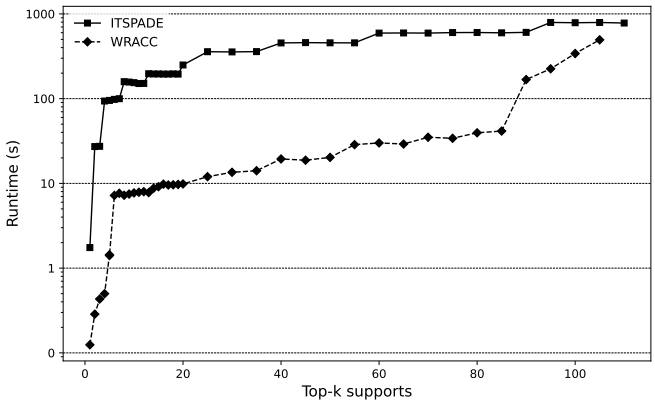

Fig. 2: CPU runtime comparison on *reuters* dataset over k values

B. Classification Accuracy

We first look at the impact of the number of fold considered for the *K-Fold Cross-Validation* over the much smaller dataset: *Protein*.

Figure 3 shows that increasing the number of fold holds little to none impact on accuracy, but increases significantly the variance of the classification. We observed the same behavior with the implementation with our *WRAcc-based* miner. Therefore, we selected a fold value at 4 for the rest of the experiment as it compromises the best between accuracy, variance, and speed.

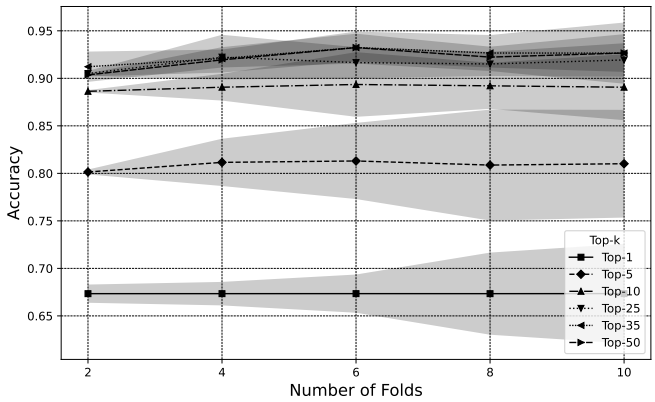

Fig. 3: *Iterative SPADE* fold accuracy on *protein* dataset for k values

Figures 4 and 5 show that the classification accuracy consistently improves as the number of top-k patterns increases, with a significant head-start for *WRAcc* in lower k values. However, the performance gain tends to plateau for both algorithms beyond a certain value of k, suggesting that a relatively small subset of highly informative patterns is sufficient to achieve very high accuracy, similar in value for both algorithms. This insight is particularly valuable in practice, as it implies that competitive classification results can be obtained without mining an excessive number of patterns—substantially reducing computation time and memory usage. Nonetheless, the observed variance is not negligible, which might hinder classification performance if not addressed.

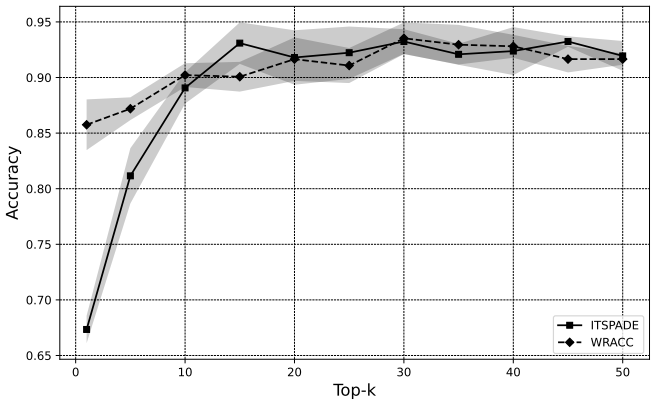

Fig. 4: Classification accuracy evolution over k values for *protein* dataset

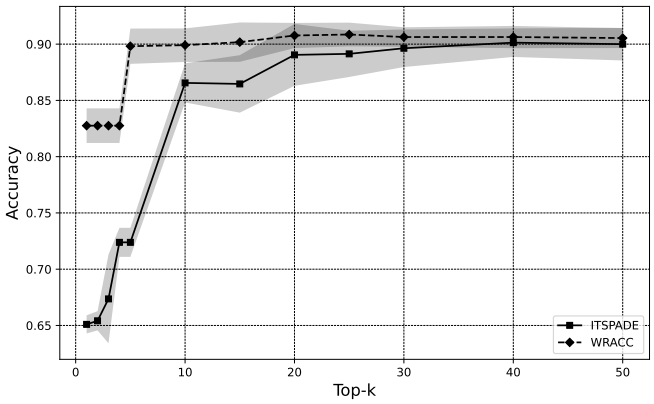

Fig. 5: Classification accuracy evolution over k values for *reuters* dataset

V. Discussion

Our results reveal distinct behaviors among the implemented sequence mining algorithms.

CM-SPADE, though designed to prune using co-occurrence patterns, struggles on high-dimensional datasets like *Reuters* due to the costly overhead of building and applying the co-occurrence matrix without deeper pruning mechanisms.

In contrast, *IT-SPADE* performs consistently well. Its iterative deepening and caching strategy enables efficient discovery of top-k patterns while avoiding unnecessary exploration, making it both fast and interpretable.

WRAcc, focused on finding discriminative rather than frequent patterns, achieves strong classification accuracy at low k values. However, beyond certain thresholds (e.g., $k > 60$ for *Protein*), it suffers from performance breakdowns. These are not due to missing pruning logic—as it does apply a WRAcc-based threshold—but rather to the massive number of candidate sequences generated and scored during its exhaustive DFS traversal. In sparse datasets, this overwhelms memory and compute resources, making the approach infeasible without further optimizations.

However, for the same k, *WRAcc* achieves the top accuracy plateau (i.e. $\pm 90\%$) when *IT-SPADE* is up to 17% less accurate.

VI. Conclusion

In this project, we implemented and evaluated several top-k sequence mining algorithms: *SPADE*, *CM-SPADE*, *IT-SPADE*, and a *WRAcc*-based miner. Each algorithm was assessed in terms of runtime efficiency, scalability, and classification accuracy across datasets .

Our experiments demonstrated the robustness of *IT-SPADE*, whose adaptive support threshold and caching strategies enabled efficient top-k extraction, even in large and sparse datasets. *CM-SPADE*, while conceptually elegant, was hindered by the overhead of its co-occurrence matrix, particularly in high-dimensional settings. Finally, *WRAcc* showed competitive performance in classification tasks for small k values, highlighting the benefit of discriminative pattern mining. However, its lack of adaptivity and exhaustive search strategy significantly limited its scalability.

Overall, this report confirms the importance of adaptivity and pruning strategies in scalable pattern mining. Future work could explore hybrid approaches, combining frequency-based pruning with discriminative scoring functions, leveraging data-driven heuristics to better guide the search in sparse, high-dimensional environments, or parallelization implementation by transforming the depth-first-search approach of *SPADE* variants to a breath-first-search.

References

[1] M. J. Zaki. SPADE: An Efficient Algorithm for Mining Frequent Sequences. In *Machine Learning*, vol. 42, no. 1–2, pp. 31–60, Jan. 2001.

[2] P. Fournier-Viger, A. Gomariz, M. Campos, and R. Thomas. Fast Vertical Mining of Sequential Patterns Using Co-occurrence Information. In *Proc. Pacific-Asia Conf. on Knowledge Discovery and Data Mining (PAKDD)*, Tainan, Taiwan, vol. 8443, *Lecture Notes in Computer Science*, pp. 40–52, May 2014.

[3] N. Lavrač, P. Flach, and B. Zupan. Rule Evaluation Measures: A Unifying View. In *Lecture Notes in Artificial Intelligence*, Vol. 2821, Springer, pp. 174–185, 2004.

[4] H. Dinkel, C. Chica, A. Via, C. M. Gould, L. J. Jensen, T. J. Gibson, and F. Diella. Phospho.ELM: a database of phosphorylation sites—update 2011. *Nucleic Acids Research*, vol. 39, no. suppl_1, pp. D261–D267, Jan. 2011.

[5] D. D. Lewis. Reuters-21578 Text Categorization Test Collection. 1997. [Online]. Available: http://www.daviddlewis.com/.../reuters21578